

Set 1 follow-ups: Core concepts

1. A pointer stores an address. What does that address identify in a real program?
2. What problem occurs if the owner finishes using allocated memory but never releases it?
3. Why is `p` dangling after `free(a)` even though `p` still contains an address?
4. Why does `a = NULL` not automatically change `p` to `NULL`?
5. Name the exact statements between which `*p` is valid.
6. If `size = 3`, which array positions are meaningful?
7. Can capacity be larger than size? Give one example.
8. Give one state that violates $0 \leq \text{size} \leq \text{capacity}$ and explain the danger.
9. What could happen if searching continues to capacity instead of size?
10. Why does doubling capacity fail when capacity begins at zero?
11. Trace what happens to the original block when temporary `realloc` returns `NULL`.
12. What value could be lost if insertion shifted from left to right?
13. Why is left-to-right shifting safe during deletion?

14. Why is index size valid for insertion but invalid for normal element access?
15. What relationship could be lost if student IDs, names, and scores were stored separately?
16. Rewrite `p->score` without using the arrow operator.
17. With `const Student *p`, what can and cannot the function change?
18. Name three checks required before committing a parsed record.
19. If one malformed line is rejected, what collection state must remain unchanged?
20. Explain “p is dangling” using claim, evidence, and consequence.

Set 2 follow-ups: State tracing

21. Does `*p += 5` change an address or a stored value? Show the state change.
22. After `free(a)`, which pointers become invalid: only a or every pointer into the block? Why?
23. If `p = &a[1]`, explain why assigning through `*p` changes `a[1]`.
24. Give an example of a function changing caller data through a pointer parameter.
25. What should the program do if `malloc` returns `NULL`?

26. For size 3 and capacity 5, should indexes 3 and 4 be displayed? Why?
27. List the safe steps for appending when the collection is full.
28. Trace the movements required to insert 15 at index 1 in [10, 20, 30].
29. Trace deletion of index 1 from [10, 15, 20, 30].
30. Why might the deleted value still appear in memory after `size--`?
31. For insertion at index 1 with size 3, what values does `i` take?
32. Why does the deletion loop stop at `size - 1`?
33. Which assignment must not execute when `tmp == NULL`?
34. Why is returning an index more useful than returning only "found"?
35. When would `s.score` be correct instead of `s->score`?
36. Give an example showing how swapping only scores corrupts student information.
37. Why does `%29[ ^ | ]` use 29 when the name array has 30 characters?
38. Why should a two-field line not be accepted with a default third value?

- 39. Are scores 0 and 100 valid? Explain the boundary rule.
- 40. Why must `fclose` be checked even after every `fprintf` succeeds?

Set 3 follow-ups: Defect diagnosis

- 41. Repair `capacity *= 2` so that zero capacity can grow.
- 42. Give a concrete inconsistent state caused by updating capacity before successful resizing.
- 43. Show why direct assignment from `realloc` can create a memory leak.
- 44. What unsafe statement might execute if a failed `realloc` is not checked?
- 45. With `size = 3` and `index = 1`, evaluate `size < index`. What happens to the loop?
- 46. Why should index validation occur before attempting allocation?
- 47. Give a state where `data[index] = value` writes outside allocated memory.
- 48. If `size++` happens before allocation fails, which invariant or meaning is violated?
- 49. Give an example of a stale value in an unused slot causing a false search result.
- 50. Which deletion indexes are valid when `size` is zero?

51. What could happen if both the owner and an alias call `free`?
52. How would you test whether repeated operations create a memory leak?
53. Why is use-after-free dangerous even when the program sometimes prints the expected value?
54. Give an example of partial corruption caused by parsing directly into `records[size]`.
55. Which operation becomes ambiguous if duplicate student IDs are accepted?
56. What should a sorting swap contain to preserve record identity?
57. Explain the type error in `p.score` when `p` is `Student *`.
58. Explain the type error in `s->score` when `s` is a `Student` object.
59. What happens to the old file immediately after opening it with `"w"`?
60. Describe a safer replacement strategy than writing directly over the original file.

Set 4 follow-ups: Failure and testing

61. For size 4, name two invalid insertion indexes that test opposite boundaries.

62. What result should inserting into an empty zero-capacity array produce?
63. After forced `realloc` failure, which four values or states should you compare?
64. Why is inserting into the middle of a full array stronger than only appending?
65. How many elements move when inserting at index 0 into an array of size 5?
66. How many existing elements move when inserting at index `size`?
67. After deleting the only element, must capacity also become zero?
68. How can you prove an invalid-index operation left the array unchanged?
69. Why should a failed resize return before executing the shift loop?
70. Which action is the final commit when loading a valid record?
71. If a duplicate line is rejected, should the accepted/rejected counters change?
72. Which required field is absent from 102 | Sokha?
73. How does an extra `%c` help distinguish valid input from 104 | Vannak | 81 | extra?
74. Why are zero and negative IDs rejected?
75. Give one value just below and one just above the valid score range.

- 76. What could happen if an unbounded name is read into `char name[30]`?
- 77. How should the program distinguish a missing input file from an empty input file?
- 78. When is append mode appropriate, and why may it be wrong for complete replacement?
- 79. Why should an audit trail record the rejection reason, not only the line number?
- 80. Give a normal test that might pass even when the boundary behavior is incorrect.

Set 5 follow-ups: Integrated review

- 81. How can you identify the owner when several pointers reference the same dynamic array?
- 82. Why may an interior pointer become invalid after a successful `realloc`?
- 83. After the first successful insertion from zero capacity, what are valid size and capacity values?
- 84. Explain why insertion allows `k == size`.
- 85. Explain why deletion rejects `k == size`.
- 86. At which step does insertion first modify the logical collection?
- 87. Why must ownership be included in the state preserved after failed resizing?
- 88. Why is `-1` a useful not-found result for an index-returning search?

89. Show how the returned index can be used to update a student's score.
90. What invariant must still hold after sorting Student records?
91. At what point in the loading pipeline does untrusted text become official collection state?
92. What temporary fields would you use when parsing a Student line?
93. Before accepting `101 | Dara | 78.5`, what must be checked besides its format?
94. If the loader silently ignores `|extra`, what validation promise is broken?
95. Why should the success message be printed only after `fclose` returns success?
96. How many times should the dynamic record block be freed, and by whom?
97. What could you do to aliases after freeing the owner to reduce accidental misuse?
98. Give a before-and-after state table that proves a failed insertion changed nothing.
99. Turn "I think the pointer is unsafe" into an evidence-based statement.
100. Apply the seven-step explanation sequence to one failed insertion.